

MondoShawan: A Post-Quantum BlockDAG with Native EVM Execution

Technical Whitepaper — v0.3 (April 2026)

Author: David Cooper, Mondoshawan Project

Testnet Explorer: <https://explorer.mondoshawan.io>

Public Repository: <https://github.com/dev-mondoshawan/mondoshawan-core-concepts>

Abstract

MondoShawan is a Layer 1 blockchain protocol that unifies three independently proven technologies into a single, production-tested architecture: the GhostDAG consensus mechanism for high-throughput parallel block production, the Ethereum Virtual Machine (EVM) for smart contract execution, and NIST-standardized post-quantum cryptography for long-term security. The protocol introduces TriStream Proof-of-Work, a novel mining architecture that simultaneously targets GPU, ASIC, and ZK-Proving hardware, distributing mining rewards across hardware classes to resist centralization. A live multi-node testnet is operational between production datacenters, producing blocks at a median interval of 10–11 seconds with real cryptographic work, a fully functional EVM RPC surface compatible with MetaMask, and peer connections secured by ML-KEM-768 (CRYSTALS-Kyber) post-quantum key exchange. This paper describes the technical design, implementation decisions, and the rationale behind each architectural choice.

1. Motivation: Why Build Another Layer 1?

The honest answer is that no existing Layer 1 does all three of these things at once.

Kaspa is the fastest PoW BlockDAG ever built. After its Crescendo hard fork in 2025, it processes 10 blocks per second with sub-second confirmation latency. It is a remarkable engineering achievement. But Kaspa has no EVM, no smart contract execution, and no post-quantum cryptography. It is a pure value-transfer network.

Ethereum has the most mature smart contract ecosystem in existence. But it abandoned Proof-of-Work, relies on a single-chain architecture with known throughput ceilings, and its cryptographic primitives (secp256k1 ECDSA) are vulnerable to a sufficiently powerful quantum computer.

The Quantum Resistant Ledger (QRL) is the most serious post-quantum blockchain project in production today. It uses XMSS signatures and has been quantum-resistant since 2018. But it has no EVM, no BlockDAG, and no smart contract ecosystem of meaningful scale.

The gap in the market is clear: there is no Layer 1 that combines the throughput of a BlockDAG, the developer accessibility of the EVM, and the security of post-quantum cryptography. MondoShawan is that protocol.

This is not a theoretical claim. The testnet is live. The code is written in Rust. The blocks are syncing between datacenters right now. The rest of this paper explains exactly how it works.

2. The Quantum Threat: Why Now?

The cryptographic foundations of Bitcoin and Ethereum — specifically the secp256k1 elliptic curve and the ECDSA signature scheme — are secure against classical computers. Against a sufficiently powerful quantum computer running Shor's algorithm, they are not.

The most dangerous near-term threat is what the security community calls “Harvest Now, Decrypt Later” (HNDL). An adversary with sufficient storage capacity can record encrypted blockchain traffic today — peer gossip, transaction broadcasts, block propagation — and decrypt it retroactively once a capable quantum computer becomes available. The Federal Reserve published a working paper in 2025 explicitly analyzing this risk for distributed ledger networks [1].

The timeline is uncertain, but the trajectory is not. IBM’s quantum roadmap targets fault-tolerant systems capable of breaking 2048-bit RSA by the late 2020s. The National Security Agency has already mandated that all US government systems migrate to post-quantum cryptography by 2030. The European Union has issued similar directives.

In August 2024, the National Institute of Standards and Technology (NIST) finalized the first three post-quantum cryptographic standards [2]:

- **FIPS 203** (ML-KEM, based on CRYSTALS-Kyber): Key encapsulation
- **FIPS 204** (ML-DSA, based on CRYSTALS-Dilithium): Digital signatures
- **FIPS 205** (SLH-DSA, based on SPHINCS+): Stateless hash-based signatures

MondoShawan implements ML-KEM-768 (FIPS 203) for all peer-to-peer key exchange, making it one of the first Layer 1 protocols to adopt the finalized NIST standard in production networking code.

3. Architecture Overview

MondoShawan is structured in four distinct layers, each with a well-defined responsibility boundary:

Layer	Components	Responsibility
Network	QUIC transport, ML-KEM-768, Noise Protocol	Encrypted peer discovery, block gossip, IBD sync
Consensus	GhostDAG, Hybrid Sled Storage	Block ordering, finality, DAG persistence
Mining	TriStream PoW, B3MemHash, ZK Prover	Block production, difficulty adjustment
Execution	SputnikVM EVM, JSON-RPC, Account Abstraction	Smart contracts, user transactions, wallet logic

These layers communicate through well-defined interfaces. The EVM execution layer submits finalized transaction receipts to the consensus layer. The mining layer

proposes new blocks to the consensus layer. The network layer propagates both. This separation of concerns allows each component to be upgraded independently.

4. Consensus: GhostDAG

4.1 The BlockDAG Model

Traditional Nakamoto consensus treats concurrent blocks as “orphans” — wasted work that does not contribute to chain security. In a network with high block rates or high latency, orphan rates can exceed 50%, meaning more than half of all mining work is discarded. This creates a fundamental tension: faster block times mean more orphans, which means less security per unit of hash rate.

GhostDAG resolves this tension by treating concurrent blocks as valid DAG nodes rather than orphans. Every block can reference multiple parent blocks, forming a Directed Acyclic Graph. The protocol then applies the GHOSTDAG algorithm to assign a “blue score” to each block, producing a total ordering of the DAG that is consistent across all honest nodes.

The key parameter is **K**, the maximum number of blocks that can be “blue” (included in the main chain ordering) at any given DAG depth. In MondoShawan, K defaults to 4, matching Kaspa’s production configuration. A higher K increases parallelism but slows convergence; a lower K converges faster but reduces throughput.

4.2 Blue Score Calculation

The blue score of a block is calculated via a BFS traversal from the genesis block. For each block, the blue score equals the maximum blue score among its blue parents, plus one. Blocks that fall outside the K-nearest-to-the-selected-chain are classified as “red” and excluded from the canonical ordering.

In practice, this means that even in a network with high latency and many concurrent miners, the vast majority of blocks contribute to chain security. The hash rate of “red” blocks is not wasted — it still raises the cost of a 51% attack.

4.3 Hybrid Storage Architecture

A naive GhostDAG implementation keeps the entire DAG in RAM. This is fine for a testnet with a few thousand blocks, but it is a production disaster. At 10 blocks per second for a year, you have over 300 million blocks. No server has that much RAM.

MondoShawan uses a hybrid storage architecture built on `sled`, an embedded key-value database:

The “hot cache” keeps the most recent 1,000 blocks in RAM for instant blue score calculation. As the DAG grows, blocks that accumulate more than 100 confirmations are pruned from the hot cache and persisted to disk. Consensus state (blue set, blue scores, block ordering) is checkpointed to disk every 100 blocks, allowing the node to restart without replaying the entire chain history.

This architecture was validated on the live testnet. Storage benchmarks using the Criterion framework demonstrated 7x performance headroom over the target block rate, confirming that the hybrid approach scales well beyond current testnet conditions.

5. TriStream Proof-of-Work

5.1 The Centralization Problem

Every successful PoW cryptocurrency eventually faces the same problem: ASIC manufacturers build specialized hardware that is orders of magnitude more efficient than consumer CPUs and GPUs. This concentrates mining power into a small number of industrial operations, reducing the decentralization that makes PoW valuable in the first place.

The standard response is to use “ASIC-resistant” algorithms like Ethash or RandomX. But ASIC resistance is a temporary measure, not a permanent solution. Given sufficient economic incentive, manufacturers will always build ASICs for any algorithm.

MondoShawan takes a different approach: instead of fighting ASICs, we embrace them — but we also embrace GPUs and ZK provers simultaneously. By running three parallel mining streams with different hardware profiles, we distribute mining rewards across hardware classes rather than concentrating them in one.

5.2 Stream A: Blake3 (ASIC-Optimized)

Stream A uses the Blake3 hash function, which is intentionally designed to be fast on all hardware, including ASICs. Industrial miners with dedicated Blake3 ASICs will dominate this stream, and that is by design. ASIC miners provide massive, committed hash rate that is expensive to acquire and difficult to repurpose. They are long-term stakeholders in the network's security.

Stream A targets a 10-second block interval. The difficulty is adjusted using the DAA described in Section 5.5.

5.3 Stream B: B3MemHash (CPU/GPU)

Stream B uses `B3MemHash`, a custom memory-hard hash function designed to level the playing field for consumer hardware. The algorithm allocates a 256KB working buffer and performs multiple rounds of Blake3 hashing and memory mixing. Because the bottleneck is memory bandwidth rather than raw compute, the advantage of a dedicated ASIC is dramatically reduced.

The memory mixing step is the performance-critical inner loop. MondoShawan implements SIMD-accelerated mixing with automatic runtime dispatch:

- **AVX2:** Processes 32 bytes per `_mm256_xor_si256` instruction ($8\times$ scalar throughput)
- **SSE2:** Processes 16 bytes per `_mm_xor_si128` instruction ($4\times$ scalar throughput)
- **Scalar:** Processes 8 bytes per `u64` XOR word (always available)

The dispatch uses Rust's `is_x86_feature_detected!` macro, which reads CPU feature flags at runtime without requiring a specific compile target. This means a single binary runs optimally on any x86-64 processor, from a decade-old laptop to a modern EPYC server.

5.4 Stream C: ZK Proving

Stream C is not traditional mining. Instead of iterating nonces, validators in Stream C generate Groth16 Zero-Knowledge proofs of state transitions over the BN254 elliptic curve. These proofs verify that a batch of transactions was executed correctly without revealing the transaction details.

Stream C is designed for ZK-proof hardware: high-end GPUs and FPGAs that can generate proofs efficiently. It earns 20% of block rewards on mainnet, creating a sustainable economic incentive for ZK infrastructure operators to participate in the network.

On the current testnet, Stream C proofs are generated and soft-verified (invalid proofs are logged but do not block consensus). Full hard verification is planned for the mainnet release.

5.5 Difficulty Adjustment Algorithm (DAA)

A multi-stream DAG requires a DAA that is both responsive and stable. Too responsive, and the difficulty oscillates wildly when hash rate fluctuates. Too conservative, and block times drift far from their targets during hash rate changes.

MondoShawan's DAA uses fixed-point arithmetic (scaling by 1,000) to avoid floating-point operations in consensus-critical code. Floating-point results vary across CPU architectures, which can cause consensus failures in a distributed network. Fixed-point arithmetic is deterministic on all hardware.

The adjustment ratio is clamped between **0.667×** and **1.5×** per epoch. This means difficulty can never more than double or halve in a single adjustment, preventing the oscillation that plagues many early-stage PoW networks. A moving-average variant averages the last N block times before calculating the adjustment, providing additional stability during periods of volatile hash rate.

This DAA was tuned live on the testnet. Initial parameters caused oscillation between difficulty levels, resulting in block times that swung between 5 seconds and 30 seconds. After tuning `INITIAL_DIFFICULTY_A = 20` and `MAX_DIFFICULTY = 28`, the testnet stabilized at a median block time of 10–11 seconds between two datacenter nodes.

6. Post-Quantum Cryptography

6.1 ML-KEM-768 Key Exchange

Every peer connection in MondoShawan is secured using ML-KEM-768 (CRYSTALS-Kyber), the NIST FIPS 203 standard for post-quantum key encapsulation [2]. This provides forward secrecy against quantum adversaries: even if a future quantum computer breaks the long-term identity keys of a node, it cannot decrypt historical session traffic.

The handshake protocol runs over a dedicated QUIC stream immediately after TLS 1.3 connection establishment:

```
Initiator                                Responder
|                                        |
|--- public_key (1184 bytes) ----->|
|<-- public_key (1184 bytes) -----|
|                                        |
|--- ciphertext (1088 bytes) ----->| (encapsulated shared secret)
|<-- ACK (1 byte) -----| (decapsulation confirmed)
|                                        |
Both sides hold the same 32-byte SessionKey
```

The resulting 32-byte session key is used to initialize an AES-256-GCM cipher for all subsequent block gossip and mempool synchronization. The ML-KEM-768 parameter set provides 128-bit post-quantum security (equivalent to 256-bit classical security), as specified in NIST FIPS 203.

6.2 TOFU Identity and Sybil Resistance

Node identities in MondoShawan are tied to Ed25519 public keys embedded in self-signed TLS certificates. When a node first connects to a peer, it records the peer's public key fingerprint (using the SHA-256 hash of the certificate's SubjectPublicKeyInfo). On subsequent connections, the node verifies that the fingerprint matches. This "Trust On First Use" (TOFU) model prevents IP spoofing and man-in-the-middle attacks without requiring a central certificate authority.

To resist Sybil attacks and eclipse attacks, the networking layer enforces several constraints:

- A maximum of three inbound peers are accepted from any single `/16` subnet, preventing a single operator from flooding the peer table with IP addresses from the same datacenter.
- Peers whose advertised peer lists have more than 80% overlap (measured by Jaccard similarity) are flagged as potential Sybil clusters and deprioritized.
- 70% of peer slots are reserved for outbound connections initiated by the local node, preventing an attacker from filling all slots with inbound connections they control.

6.3 Noise Protocol Encryption

In addition to the ML-KEM-768 session key exchange, all peer connections are encrypted using the Noise Protocol Framework (XX handshake pattern) via the `snow` crate. This provides an additional layer of authenticated encryption using classical cryptography, ensuring that connections are secure even on networks where the QUIC transport layer is compromised.

7. EVM Execution

7.1 SputnikVM Integration

MondoShawan integrates the Ethereum Virtual Machine using SputnikVM (`evm` crate, version 0.41), configured for the Shanghai hard fork. The Shanghai configuration includes support for EIP-3855 (`PUSH0` opcode), EIP-3860 (initcode size limits), and the full set of EIPs included in the Merge and subsequent upgrades.

Any Solidity contract compiled for Ethereum's Shanghai fork deploys and executes identically on MondoShawan. The EVM state is managed through a custom `EvmState` struct that wraps `Arc<RwLock<HashMap>>` for thread-safe concurrent access to account balances, nonces, and contract bytecode.

7.2 JSON-RPC and gRPC Surfaces

MondoShawan exposes two external interfaces for interacting with the EVM:

The **JSON-RPC v1** endpoint implements the full Ethereum JSON-RPC specification, including `eth_blockNumber`, `eth_getBalance`, `eth_sendRawTransaction`, `eth_call`, `eth_estimateGas`, `eth_getTransactionReceipt`, and 120+ additional methods. This endpoint is fully compatible with MetaMask, Hardhat, Foundry, and other standard Web3 tooling. API key authentication and per-IP rate limiting (token bucket algorithm) are enabled by default.

The **gRPC v2** endpoint provides a binary-native alternative to JSON-RPC, eliminating the overhead of JSON serialization and hex encoding. Internal benchmarks show a 3.3× throughput improvement over the JSON-RPC path for high-frequency queries. The gRPC surface implements the same method set as JSON-RPC but uses Protocol Buffers for serialization.

7.3 Account Abstraction

MondoShawan implements native Account Abstraction (AA) at the protocol level, extending the standard EVM account model with programmable wallet logic.

The AA system supports five wallet types:

- **Basic:** A standard programmable wallet with a single owner key.
- **MultiSig:** Requires M-of-N signatures from a defined signer set before executing transactions.
- **SocialRecovery:** Allows a set of trusted guardians to collectively recover a wallet if the owner's key is lost.
- **SpendingLimit:** Enforces daily, weekly, and monthly spending caps, with per-address limits for fine-grained control.
- **Combined:** A full-featured wallet combining multi-sig, social recovery, and spending limits.

This is implemented natively in the protocol layer rather than as a smart contract standard (like ERC-4337), which means it does not require a separate bundler network or relayer infrastructure.

8. Sharding

MondoShawan's sharding layer implements horizontal partitioning of the transaction state. The current implementation (Phases 1–5 complete) supports consistent-hash-based address routing, receipt-based cross-shard transaction processing (inspired by Near Protocol's approach), exponential backoff retry with configurable timeout and refund, and a Write-Ahead Log (WAL) for cross-shard message durability.

Phase 6 (in progress) adds full state synchronization between shards, transaction routing from the main chain to shards, and a catch-up protocol for shards that fall behind.

The sharding layer uses `tokio::sync::RwLock` for per-shard state isolation, with a documented lock acquisition order to prevent deadlocks. Transaction pools are bounded at 50,000 transactions per shard (DoS protection), with FIFO eviction when the limit is reached.

9. Tokenomics

9.1 QMON Token

The native token of the MondoShawan network is **QMON**, with a hard maximum supply of **1,000,000,000 (1 Billion) tokens**.

The supply is divided between mining emissions and a genesis allocation:

Allocation	Percentage	Tokens	Vesting
TriStream Mining	70%	700,000,000	~30 years via block rewards
Ecosystem & Grants	8%	80,000,000	6-month cliff, quarterly over 4 years
IKO Public Sale	7%	70,000,000	20% at TGE, linear over 5 months
Team & Founder	5%	50,000,000	12-month cliff, linear over 24 months
Development Fund	4%	40,000,000	Milestone-based
Liquidity	3%	30,000,000	Locked ≥ 12 months
Marketing & Community	2%	20,000,000	Monthly over 18 months
Treasury Reserve	1%	10,000,000	12-month lock, then governance-controlled

At the Token Generation Event (TGE), only **1.4% of total supply** is immediately circulating (20% of the 7% IKO allocation = 14,000,000 QMON). All other genesis tokens are locked or vesting.

9.2 Emission Schedule

Mining emissions follow a Kaspa-style chromatic decay. The monthly reduction factor is $(\frac{1}{2})^{(1/12)} \approx 0.9439$, meaning rewards decrease by approximately 5.6% each month. This produces an effective annual halving without the sudden economic shock of a traditional 4-year halving event.

Approximately 50% of the mining supply will be emitted by year 4, 90% by year 10, and 99% by year 20. The final 1% trickles out over the following decade.

9.3 Fee Economics

Transaction fees are split equally between miners (50%) and a burn address (50%). This creates a deflationary pressure that increases with network usage: as transaction volume grows, the burn rate grows with it, reducing the effective circulating supply over time.

10. Current Status and Roadmap

10.1 Testnet Status (April 2026)

The following components are fully operational on the live testnet:

- Real PoW mining with Blake3 and B3MemHash, difficulty adjustment, parallel mining threads
- GhostDAG consensus with hybrid hot/cold storage and checkpoint save/load
- Multi-node P2P synchronization verified across production internet (VPS deployment)
- Noise Protocol encryption on all peer connections
- Full EVM execution with contract deployment and state persistence
- JSON-RPC and gRPC endpoints with API key authentication and rate limiting
- State snapshots (create, list, restore via RPC)
- Account Abstraction wallet system

The testnet is hosted on production infrastructure (Hostinger VPS, 4 CPU, 16GB RAM) with $\frac{24}{7}$ uptime via systemd auto-restart.

10.2 Roadmap

Phase	Milestone	Status
Phase 1	Public testnet at explorer.mondoshawan.io	Complete
Phase 2	Desktop wallet (Tauri/React)	In progress
Phase 3	External security audit	Planned
Phase 4	Code freeze (v0.1.0-alpha)	Planned
Phase 5	IKO on Kommunitas (Polygon ERC-20)	Planned
Phase 6	Mainnet launch with TriStream	Planned
Phase 7	GPU mining via OpenCL	Planned
Phase 8	Sharding Phase 6 completion	Planned

11. Competitive Analysis

MondoShawan occupies a unique position in the Layer 1 landscape. No existing protocol combines all three of its core pillars simultaneously:

Protocol	BlockDAG	Post-Quantum Crypto	EVM	PoW
MondoShawan	GhostDAG	ML-KEM-768 (FIPS 203)	SputnikVM	TriStream
Kaspa	GhostDAG	None	None	Blake3
Ethereum	None	None (planned)	Native	None (PoS)
QRL	None	XMSS	None	RandomX
Bitcoin	None	None	None	SHA-256

Kaspa is the most technically comparable protocol. MondoShawan can be understood as “Kaspa + EVM + post-quantum security.” The TriStream mining model is a direct evolution of Kaspa’s single-stream Blake3 mining, adding GPU accessibility and ZK-proof integration.

12. Conclusion

The blockchain industry is at an inflection point. The quantum threat is no longer theoretical — it is a matter of timing. The “harvest now, decrypt later” attack vector means that data being transmitted over classical-cryptography networks today is already at risk.

MondoShawan is not a response to a future problem. It is a production system addressing a present one. The testnet is live. The post-quantum key exchange is running between datacenters. The EVM is executing Solidity contracts. The blocks are syncing.

The combination of GhostDAG throughput, EVM developer accessibility, and NIST-standardized post-quantum security creates a protocol that is uniquely positioned for the next decade of blockchain infrastructure.

For technical review access to the full codebase, contact: **DM @DevMondoshawan**

References

- [1] Mascelli, J. et al. “Harvest Now Decrypt Later: Examining Post-Quantum Cryptography and the Data Privacy Risks for Distributed Ledger Networks.” Federal Reserve Board, 2025. <https://www.federalreserve.gov/econres/feds/harvest-now-decrypt-later-examining-post-quantum-cryptography-and-the-data-privacy-risks-for-distributed-ledger-networks.htm>
- [2] National Institute of Standards and Technology. “FIPS 203, Module-Lattice-Based Key-Encapsulation Mechanism Standard.” August 13, 2024. <https://csrc.nist.gov/pubs/fips/203/final>
- [3] National Institute of Standards and Technology. “NIST Releases First 3 Finalized Post-Quantum Encryption Standards.” August 13, 2024. <https://www.nist.gov/news-events/news/2024/08/nist-releases-first-3-finalized-post-quantum-encryption-standards>
- [4] Kaspas. “Kaspas Blockchain — GhostDAG Protocol.” <https://kaspas.org>

[5] MarketsandMarkets. “Post-Quantum Cryptography Market Report 2025.”
<https://www.marketsandmarkets.com/Market-Reports/post-quantum-cryptography-market-126986626.html>